

Third Lab - Program 0.5.0

For the next iteration of your route reader program, you'll add in the use of a geolocation service to get global location information for all the IPs encountered in your traces. That data will then be combined with your trace data in preparation for visualization of the routes.

Part 1: Setup

Before you can add the new geolocation feature to your project, you'll need to get a few things set up. First, the API is free but requires authentication. You need to make an account. Second, the feature requires several new packages. You need to use `uv` to add these as dependencies for your project.

1. Sign up for a free-tier account at <https://app.ipgeolocation.io/signup>. The free tier offers 1000 IP queries per day. Once you've made your account, sign in and find your way to the account dashboard. There you'll see an API key. You'll need that soon.
2. Add the following dependencies to your project: `pandas requests requests-cache dot-env`
 - [pandas](#) will make working with our traceroute data a bit easier. It also integrates well with the visualization library we're going to be using.
 - [requests](#) and [requests-cache](#) let you make HTTP requests to the API and cache the results. Caching can help you manage your free-tier quota and avoid duplicate API calls.
 - [dot-env](#) streamlines working with **environment variables** and a project-level `.env` environment variables file, which is part of best practices for using authentication tokens and passwords within your source code.

Part 2: Working out the API Calls

Before you integrate the API and the new packages into your project, it's a good idea to tinker around and make sure you know how to make them work independently of your specific project needs. You can do this in a `.py` file that is separate from all your code or just temporarily add it (maybe in an `if __name__ == "__main__"` block) to existing files. The point is to write small blocks of code that print or otherwise do something to confirm that the new package methods are doing what you need them to do.

Authentication

The API documentation actually gives you a [bit of starter code](#). What you should notice is the inclusion of `API_KEY` in the query string for the variable `api_key`. The wrong way to handle this is to literally paste your key into your code. The right way to do this is to store your key as an environment variable in a file called `.env`. You can then use the `dot-env` package to make [reading that file](#) into your code a snap. Alternatively, [uv can load your .env file at the command line](#). In either case, you can access your system environment variables from within your Python code through the [os module](#).

Now, let's make sure you can set up and access a `.env` file.

1. Create `.env` (in the project root) and put your API key in there: `API_KEY=...`
2. Add `.env` to your `.gitignore`. Just list it on a line by itself. Put it anywhere.
3. Experiment with code to make sure you can load and read your key (print the key?)

Once you know you can “load” the key into your program, then you can focus on making API calls.

Caching Requests

Your program might need to make a lot of calls to the API, and some of them might be for the same IP. That’s going to eat into your quota and slow down your program. To avoid both situations, you can run a local **cache** for your requests using `requests-cache`. Any request made will first check the cache to see if you already cached that request. If so, you’ll get the cached request. If not, you’ll make the request and cache the result for later.

Your application only makes requests to one API, and you can get away with simple [patching](#) rather than sessions.

Now, let’s make some API calls that include an authentication token and make use of a request cache.

1. Expand your experimental code to make an API query and pretty-print the resultant [JSON](#). This query must make use of *query parameters*. While you could try to construct the request as `str` data, a better approach is to let `requests` do the work for you through the [params](#) argument.
2. Check out the section of the [API Docs](#) called *Get the Required Fields Only (fields Parameter)* and modify the query to only retrieve country, city, latitude, and longitude. This is the query you’re going to need for the actual project.

You can now make queries to your API, and you’ve worked out the gist of the specific query needed by your project.

Pandas

[Pandas](#) is the workhorse of most data-driven projects. Its [DataFrame](#) type holds flat data and provides a wealth of functionality for analyzing, transforming, and otherwise working with that data.

1. [Read your CSV into a DataFrame](#).
2. Give the [columns proper, descriptive names](#). Make sure that each column is [parsed](#) for the appropriate datatype. Print the frame.
3. [Use apply to apply a custom function to a column](#). Do something simple in the function. Print the result.
4. [Find the unique set of source IPs in your data](#). Print them.

These tasks cover the key functionality needed for your project, but you might also play around with various ways to add new columns to a DataFrame. The [concat](#) method can get the job done for adding both rows and columns, depending on the value of the `axis` argument. Now that you can do some project-related work with Pandas, you’re ready to cook.

Part 3: Building the New Feature

Your current program ends by writing the traceroute results as tidy data to a CSV. We’re going to extend it so that data is then read from the file, used to carry out geolocation searches for all the IPs encountered by the traces, and organized for eventual visualization.

Project Setup

Let’s first get organized:

- In `src/route_reader`, make a new module (file) `visualize_routes.py` and in `tests/` make a new test file `test_visualize_routes.py` to go with it.

- In `src/route_reader/main.py`, make a new function `see_main`. Rename the old `main` to `run_main`. Stub out the new function `see_main` to do a print or something.
- In `pyproject.toml`, add a new script `[project.scripts]` entry for a script called `see-routes` that runs `see_main` in `main.py`. Fix the old script for `run-routes` so that it points to `run_main`. *Test that you can still run the old script and that running the new script executes your stub.*
- In the `visualize_routes` module, create the following *stubs*. Document the purpose in a docstring.
 - `load_routes` takes the file name for your tidy data and reads that file into a `pandas` `DataFrame`. To that frame, add proper column names and make sure the columns are the most appropriate data type. It returns the cleaned-up frame.
 - `locate_hops` takes a `DataFrame` (the output of `load_routes`) and uses the API to extend that frame with four new columns: the country, city, latitude, and longitude for the destination IP for each hop/row. Return the frame.
 - `locate_sources` takes a `DataFrame` (the output of `load_routes`) and extracts from it the source IPs. It then, with help from the API, constructs a frame that has source information. Specifically, for each source: datetime of the trace, source IP, country, city, latitude, longitude. It returns this frame.
- In `test_visualize_routes.py`, add a stub test for each function. Call your function in the test, but feel free to ignore the result and just use `assert True` to stub out a test that will pass. The goal right now is structure and scaffolding, not completing feature logic.

Once you're done, you should be able to run `run-routes` and `see-routes` without any errors. The former will do what it's always done. The latter will do whatever you stubbed it out to do. Additionally, you should be able to run your tests in `test_visualize_routes.py` and see them all pass without error. Doing all of this means you have an executable foundation for building your new module code.

Programming: Building the Module, Tests-First

You should work in a test-driven manner. The extreme way to do this is to write *all* your tests first. A less extreme pattern might be to go one function at a time by writing the test, then the definition. You can decide.

- In `test_visualize_routes.py`, write unit tests for the three visualize functions. Use fixtures and monkey patching where possible and appropriate. Once a test is done, you should be able to run it and watch it fail due to the fact that the function isn't complete yet. Again, you can now proceed on an executable foundation and one that will test your work as you go.
- In `visualize_routes.py`, replace the stub with a complete definition. Use your unit tests to check your work and debug as needed.

Assuming your testing logic is sound, once all your tests are written and pass, you should be ready to complete the new `see-routes` program stub.

Stubbing Out see-routes

We don't have visualization code yet, but that doesn't mean we can't stitch together a partially feature-complete version of our program. Doing so lets us gut-check that our new module functions will work well together and in the context of the target program.

- In `main.py`, write out a program that takes one CLI argument, the file name of the tidy data file. It loads that file, locates all the hops and sources, and for now, prints both frames to the terminal.

Once this works, you're done!

The Middle Way

What the process above coached you through is a more or less *bottom-up* approach. You completed the module code before you touched the program that used that code. The bottom here refers to the small units of code from which our target program is built. By starting with these and then working on the code that combines them, we are building our program from independent to dependent units. The benefit here should be obvious. You know something works when you use it. The downside is that you can easily lose the forest for the trees. The bottom units you think you need might not actually be what you need, and you often only discover that when you try to use them in context.

A *top-down* approach would begin with `main`, the unit dependent on all other units, and work your way down. To do this, you have to utilize stubs, creativity, and a bit of optimism to declare (stub + purpose) and stitch together all the code you need down to the bottom. The upside here is that you avoid the problem of the bottom-up approach. You start with in-context coding, design the units that make sense in-context, then build them to fit that clearly identified need. The downside is that you can spend a fair amount of time building something that isn't fully testable. Until you've worked out a whole code execution path in your program, you won't have something that can be tested. You can still write tests as you go, but you're deferring verifying these tests and debugging your code until you get to the bottom.

The compromise is to work *middle-out*. Figure out some *part* of your system, usually some critical middle component, and work out how it will interact with your system. If you can clearly identify what needs it has for the units below it as well as the needs of the units above it, then you can use *fixtures* and *mocks* to fully build and test that component to specification. You actually inherit the downsides of both top-down and bottom-up approaches here, but by working on modest-sized parts of the whole system, you can mitigate the cost (having to revisit and redo some work) of misunderstanding the overall system design and how your middle chunk interacts with the units above and below it.